

Analisi e descrizione dell'applicazione “Parking Spot”

1. Introduzione e contesto dell'applicazione

Parking Spot è un'applicazione web per la gestione e la prenotazione di posti auto. Il sistema rappresenta un parcheggio composto da 87 posti, suddivisi nelle zone A e B, e permette di visualizzarne la disponibilità in una determinata fascia oraria attraverso una mappa 2D. L'utente deve registrarsi, successivamente può autenticarsi, completare il proprio profilo, associare uno o più veicoli e prenotare i posti disponibili nella fascia oraria che preferisce.

L'applicazione distingue, a livello di back-end, le funzionalità dell'utente da quelle amministrative. L'utente può consultare, creare e cancellare esclusivamente le proprie prenotazioni. L'amministratore, tramite endpoint REST protetti, può gestire i posti auto, consultare tutti gli utenti e visualizzare l'insieme delle prenotazioni. Nell'attuale front-end non è però presente una sezione amministrativa dedicata: tali operazioni sono quindi disponibili nelle API, ma non sono esposte dall'interfaccia grafica. L'autenticazione è delegata a Keycloak, mentre il back-end verifica il token JWT e applica i controlli di autorizzazione previsti per i ruoli USER e ADMIN.



Figura 1: Diagramma UML dei casi d'uso dell'applicazione.

2. Analisi dei requisiti

I requisiti funzionali principali comprendono la registrazione e il login, la gestione del profilo utente, l'inserimento dei veicoli, la ricerca dei posti disponibili per data e fascia oraria, la creazione di una prenotazione e la sua cancellazione. Il profilo è collegato all'identificativo Keycloak, l'email, il codice fiscale e la targa sono soggetti a vincoli di unicità. Le prenotazioni possono avere una durata compresa tra una e dodici ore e non possono riferirsi a una fascia oraria già trascorsa.

Tra i requisiti non funzionali rientrano la sicurezza, la consistenza dei dati e la gestione della concorrenza. Il back-end è stateless e accetta Token Bearer firmati dal realm Keycloak, le API amministrative sono

protette mediante il controllo del ruolo. Le operazioni di prenotazione verificano la sovrapposizione temporale sia sul posto auto, sia sul veicolo. Il lock pessimistico acquisito sui record del parcheggio e del veicolo, insieme ai controlli di sovrapposizione eseguiti nella stessa transazione, impedisce che due richieste concorrenti prenotino lo stesso posto o impieghino lo stesso veicolo nella medesima fascia oraria. L'entità **Parcheggio** contiene anche un campo **@Version**, utilizzato da JPA per il controllo ottimistico quando il record viene effettivamente aggiornato, la creazione di una prenotazione, tuttavia, non modifica tale record e si basa principalmente sui lock pessimistici e sulle query di sovrapposizione.

3. Back-end e tecnologie impiegate

Il back-end è sviluppato in Java 17 con Spring Boot e utilizza Spring Web MVC per le API REST, Spring Data JPA per la persistenza, Spring Validation per il controllo dei dati e Spring Security OAuth2 Resource Server per la validazione dei JWT. Il database PostgreSQL conserva utenti, veicoli, parcheggi e prenotazioni. PostgreSQL e Keycloak vengono avviati mediante Docker Compose, mentre Hibernate aggiorna lo schema a partire dalle entità JPA.

L'applicazione adotta un'architettura a livelli.

I Controller ricevono le richieste HTTP e delegano l'elaborazione ai servizi.

I Service contengono la logica applicativa e delimitano le transazioni.

I Repository incapsulano l'accesso ai dati.

Le Entity principali sono **Utente**, **Veicolo**, **Parcheggio** e **Prenotazione**. Ogni prenotazione collega un utente, un veicolo e un posto auto.

Sono inoltre utilizzati DTO di richiesta e risposta, che separano il modello persistente dai dati scambiati con il client.

Metodi HTTP esposti dalle API REST

Le API utilizzano i principali metodi del protocollo HTTP secondo le operazioni richieste sulle risorse del sistema.

Metodo	Endpoint	Funzione principale
GET	/api/auth/me; /api/parcheggi; /api/parcheggi/disponibili; /api/parcheggi/disponibilita; /api/parcheggi/{id}; /api/prenotazioni; /api/prenotazioni/admin; /api/utenti/me; /api/utenti; /api/utenti/{id}; /api/veicoli	Consultazione dell'utente autenticato, dei parcheggi, della disponibilità, delle prenotazioni, dei profili e dei veicoli. Gli endpoint amministrativi richiedono il ruolo ADMIN.
POST	/api/parcheggi; /api/prenotazioni; /api/utenti/me; /api/veicoli	Creazione di un parcheggio, di una prenotazione, del profilo personale e di un veicolo. La creazione dei parcheggi è riservata all'amministratore.
PUT	/api/parcheggi/{id}	Aggiornamento completo dei dati di un parcheggio, consentito esclusivamente all'amministratore.
DELETE	/api/parcheggi/{id}; /api/prenotazioni/{id}; /api/veicoli/{id}	Eliminazione di un parcheggio oppure cancellazione di una prenotazione o di un veicolo appartenenti all'utente autenticato.

Tabella 1: Principali metodi HTTP ed endpoint REST esposti dal back-end.

Sono inoltre presenti due endpoint **PATCH**, riservati all'amministratore:

```
PATCH /api/parcheggi/{numero}/abilita
PATCH /api/parcheggi/{numero}/disabilita
```

Tali endpoint consentono di modificare lo stato operativo di un posto auto.

Pattern architetturali e applicativi nel back-end

Nel back-end sono riconoscibili alcuni pattern architetturali e applicativi utilizzati per separare le responsabilità e ridurre l'accoppiamento tra i componenti. Non si tratta esclusivamente di design pattern GoF, ma di pattern tipici delle applicazioni enterprise.

La creazione della prenotazione è l'operazione transazionale più delicata. Il Service valida la fascia oraria, identifica l'utente dal JWT, acquisisce un lock pessimistico sul posto auto e sul veicolo e controlla eventuali sovrapposizioni. Solo al termine dei controlli viene salvata l'entità e viene effettuato il commit della transazione. In presenza di errori, l'operazione viene annullata e il client riceve uno stato HTTP coerente.



3

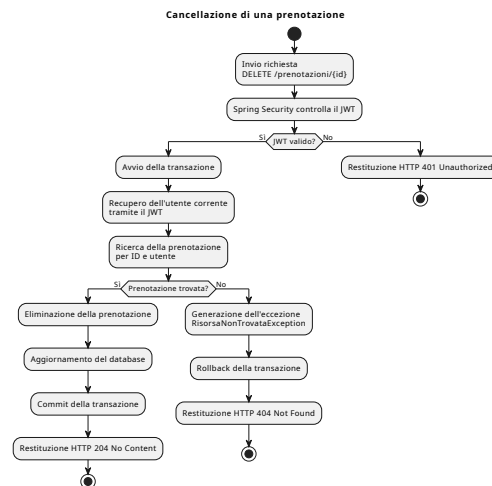


Figura 3: Activity Diagram UML della cancellazione di una prenotazione.

La gestione degli errori è centralizzata tramite `@RestControllerAdvice`, che traduce eccezioni applicative, conflitti di integrità e problemi di concorrenza in risposte HTTP strutturate. La consistenza è inoltre garantita dai vincoli di unicità, dalle associazioni obbligatorie tra le entità e dalle query che rilevano le sovrapposizioni temporali. All'avvio è presente un iniziatore che completa i posti fino al numero 87 evitando duplicati.

4. Front-end e tecnologie impiegate

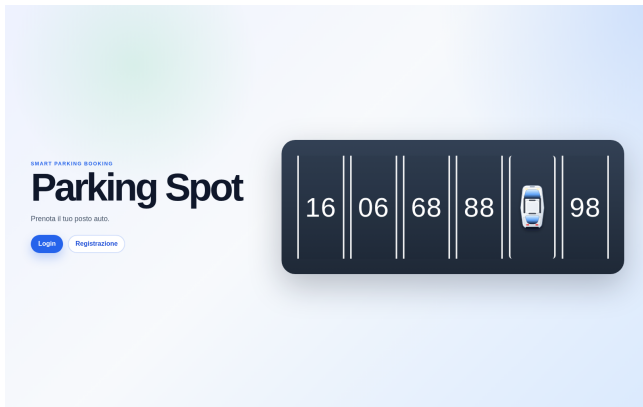
Il front-end è realizzato con Angular mediante un componente standalone. L'interfaccia utilizza `FormModule` per il binding dei moduli, `HttpClient` per le richieste REST e `RxJS` per la gestione asincrona delle risposte. La libreria `keycloak-js` avvia la sessione Single Sign-On, esegue login, registrazione e logout e rende disponibile il token da allegare alle richieste protette.

La pagina iniziale presenta l'accesso e la registrazione. Dopo aver effettuato il login, viene mostrata una dashboard composta da due sezioni: "Prenotazioni e mappa" e "Profilo e veicoli". Nella prima l'utente sceglie data, ora, durata e veicolo e la risposta del back-end viene rappresentata sulla planimetria con colori diversi per posti liberi, occupati e per le categorie grafiche "disabili" e "scarico merci". Nella seconda sezione vengono gestiti i dati anagrafici e i veicoli associati al profilo.

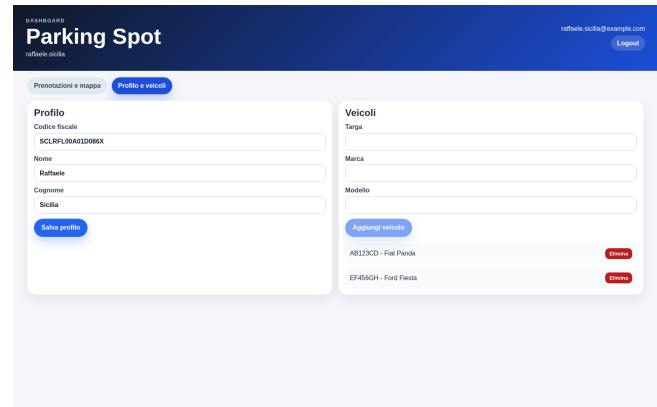
Pattern progettuali nel front-end

Nel front-end i pattern sono applicati soprattutto nei servizi Angular e nella gestione asincrona delle comunicazioni con il back-end.

Pattern	Applicazione nel progetto
Singleton	<code>KeycloakService</code> e <code>ParkingApiService</code> sono dichiarati con <code>providedIn: 'root'</code> . L'injector principale di Angular mantiene quindi una sola istanza condivisa di ciascun servizio per tutta l'applicazione.
Facade	<code>ParkingApiService</code> svolge un ruolo assimilabile a una Facade, poiché espone al componente un punto di accesso unitario alle operazioni su profilo, veicoli, parcheggi e prenotazioni, nascondendo URL, parametri e dettagli delle chiamate HTTP. Anche <code>KeycloakService</code> incapsula l'interazione con la libreria <code>keycloak-js</code> .
Interceptor	L' <code>authInterceptor</code> intercetta le richieste HTTP dirette al back-end e aggiunge automaticamente l'header <code>Authorization</code> con il Token Bearer, evitando di ripetere questa operazione in ogni chiamata.
Observer	Le operazioni di <code>HttpClient</code> restituiscono <code>Observable</code> . Il componente si sottoscrive con <code>subscribe()</code> e reagisce alla ricezione dei dati o alla presenza di errori, secondo il meccanismo del pattern Observer.



(a) Pagina di accesso e registrazione.



(b) Gestione del profilo e dei veicoli.

Figura 4: Principali schermate di autenticazione e gestione dei dati personali.

La mappa 2D è costruita a partire dai dati restituiti dall'endpoint di disponibilità. I posti vengono ordinati per numero e arricchiti sul client con zona, tipologia e posizione grafica. Le tipologie “disabili” e “scarico merci” sono quindi informazioni definite esclusivamente nel front-end in base al numero del posto, non sono memorizzate nel database e non vengono validate dal back-end. Di conseguenza, i posti rappresentati come riservati ai disabili restano prenotabili come posti ordinari, mentre quelli mostrati come area di scarico merci vengono resi non prenotabili soltanto dall'interfaccia grafica. I pulsanti “Zona A” e “Zona B” permettono di concentrare la visualizzazione su una singola area, mentre la legenda rende immediata l'interpretazione dello stato di ogni posto. Il pulsante di prenotazione viene disabilitato quando il posto non è prenotabile, non è disponibile oppure mancano profilo o veicolo.

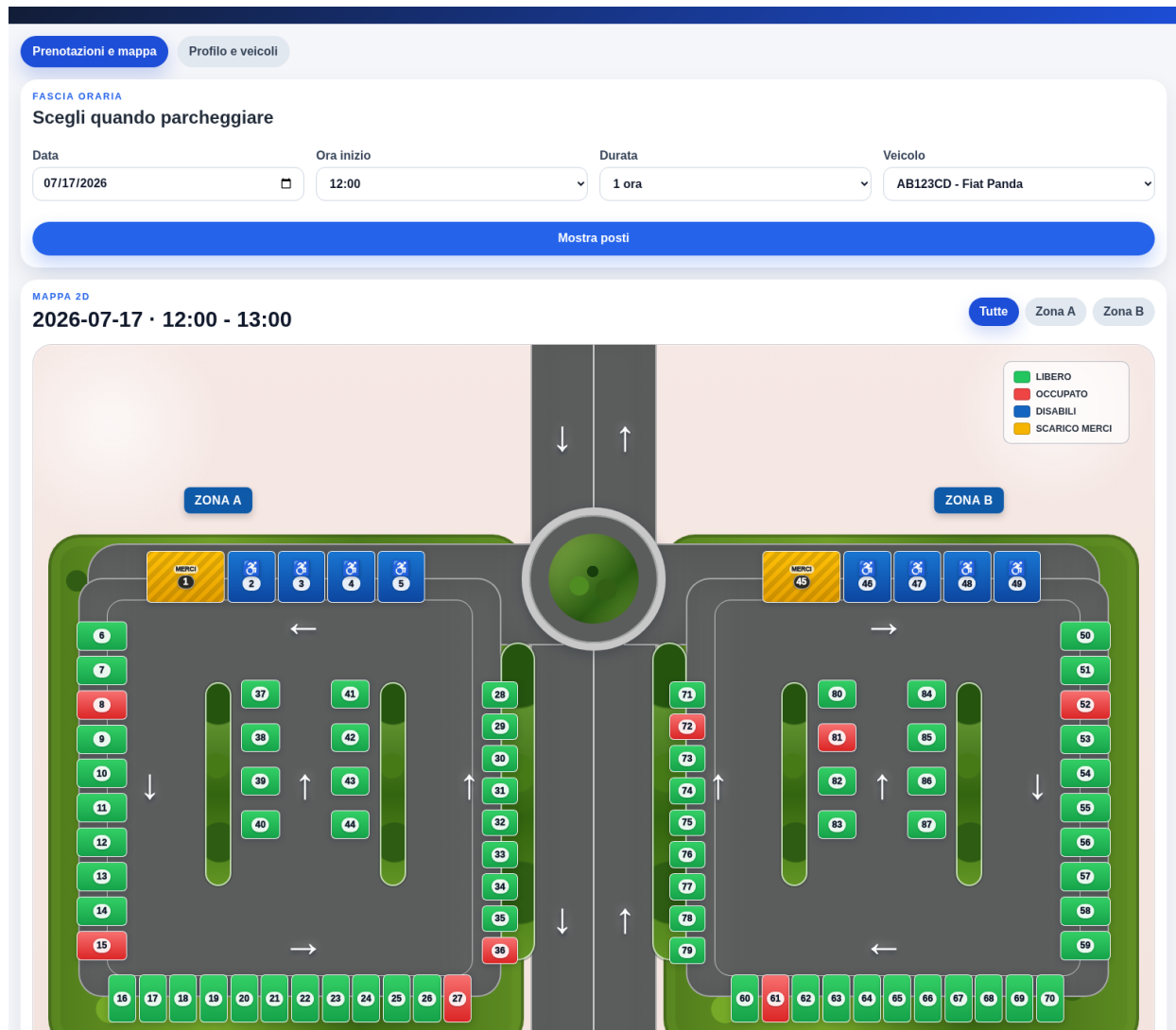


Figura 5: Ricerca della disponibilità e mappa bidimensionale del parcheggio.

5. Conclusioni

Parking Spot realizza un flusso completo di autenticazione, gestione dei dati personali e prenotazione temporale dei posti auto. La separazione tra Angular, API REST, Service, Repository, PostgreSQL e Keycloak rende il progetto modulare e facilita la manutenzione. I controlli di unicità, autorizzazione, sovrapposizione e concorrenza garantiscono una gestione coerente delle prenotazioni anche in presenza di richieste simultanee.

Gli sviluppi successivi potrebbero riguardare l'uso di HTTPS, una sezione ampia di test automatici, notifiche sulle prenotazioni, strumenti amministrativi dedicati nel front-end.